One Ciphertext, Two Views: Discordのグループ通話における映像注入攻撃

山下 勇斗[1,2,a)]　木村 隼人[2,1]　田中 篤[1]　五十部 孝典[1]

概要：Discord は音声・映像のグループ通話を提供する世界最大級のプラットフォームであり，E2EE プロトコル DAVE に AES-GCM を用いる．AES-GCM は鍵コミット性を欠き，同一の暗号文が異なる鍵の下で受理され，異なる平文へ復号され得る．既存研究は，この性質と DAVE における鍵更新の非同期性，Opus の特性を利用し，特定の受信者にのみ任意の音声を再生させる攻撃を示した．しかし，現実的な攻撃としては二つの障壁が残されている．第一に，鍵更新メッセージを識別し，標的への配送のみを選択的に破棄する必要がある．第二に，非標的における復号結果を制御できず，ノイズとして破棄される．本研究では，これら二つの障壁を取り除き，Discord に対するより現実的な攻撃を示す．第一の障壁に対しては，標的への通信を一時的に遅延させるだけで鍵の非同期状態を誘発できる攻撃を示す．第二の障壁に対しては，攻撃対象を映像へ拡張し，H.265 の構造を利用することで，単一の暗号文から，異なる鍵を持つ受信者ごとに攻撃者が意図した異なる映像を正常に再生させる映像注入攻撃を提案する．さらに，実機実験により鍵の非同期状態の持続時間を測定し，その時間内に攻撃用暗号文を生成して攻撃を実行可能であることを実証する．本脆弱性は，Discord 社への責任ある開示を経て公式に脆弱性として認定され，報奨金の付与を受けるとともに，現在，我々の提案を含む修正が検討されている．

One Ciphertext, Two Views: A Video Injection Attack in Discord Group Calls

Hayato Yamashita[1,2,a)]　Hayato Kimura[2,1]　Atsushi Tanaka[1]　Takanori Isobe[1]

Abstract: Discord protects its audio and video group calls with the E2EE protocol DAVE, which uses AES-GCM. Because AES-GCM lacks key commitment, one ciphertext can decrypt to different valid plaintexts under different keys. Prior work exploited this with DAVE asynchronous key updates and Opus to play arbitrary audio to one target, but requires dropping target key-update messages and leaves non-target output as uncontrolled noise. We instead induce the asynchronous state merely by delaying target traffic, a simpler and stealthier attack. We also extend from audio to video, exploiting H.265 so one ciphertext makes each receiver render a distinct adversary-chosen video under its own key, without noise. Experiments on real Discord clients confirm an attack ciphertext fits the measured asynchronous window. After responsible disclosure, Discord recognized the vulnerability, awarded a bounty, and is considering a fix including our proposal.

1. はじめに

近年，プライバシー保護の観点から，リアルタイム通信やメッセージングサービスにおけるエンドツーエンド暗号化（E2EE）は標準的な要件となりつつある．月間アクティブユーザー数が 2 億人を超える Discord も，2024 年に独自の E2EE プロトコルである DAVE を導入している [1]．DAVE では，鍵管理に MLS [2] を採用し，メディアパケットの暗号化には AES-GCM を用いている．しかし，AES-GCM は鍵コミット性を本来的に保証していない

[1] 大阪大学
The University of Osaka
[2] 国立研究開発法人情報通信研究機構
National Institute of Information and Communications Technology
[a)] hayato.yamashita@ist.osaka-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

ため，攻撃者は，異なる二つの鍵の双方において正常に認証され，かつそれぞれ異なる平文に復号される曖昧な暗号文を生成できる [3].

この特性を突いた先行研究として，山下ら [4] は，AES-GCM の鍵コミット性欠如を MLS の非同期な鍵更新，および音声コーデック Opus のデコード仕様と組み合わせることで，グループ通話中に特定の受信者にのみ任意の音声を再生させる音声注入攻撃を提案した．この攻撃は，Opus デコーダがペイロード末尾のパディング領域を無視する特性を利用し，タグ衝突に必要な調整ブロックをこの不可聴領域に隠蔽することで成立している．しかし，この攻撃を実環境で現実的に遂行する上では，二つの大きな課題が残されていた．

第一に，受信者間で鍵の非同期状態を強制するため，暗号化された制御トラフィックから特定の MLS Commit メッセージのみを識別して破棄する，極めて難易度の高い精密なネットワーク遮断が必要であった．第二に，非標的側の復号データは単なるノイズデータとなるため，非標的受信者に対してはパケットを破棄させて無音にすることしかできず，送信した単一の暗号文から標的と非標的の双方にそれぞれ別の意味のあるメディアを同時にデコード・再生させることは原理的に不可能であった．このため，先行研究は鍵コミット性欠如が実システム上で攻撃に利用可能であることを示した一方，現実的かつ検知困難な攻撃としての実行可能性は限定的であった．

1.1 本研究の貢献

本研究では，これらの制約を克服し，実環境において実行可能かつ検知困難な映像注入攻撃を実現する．具体的には，第一の制約に対して，標的宛ての TCP トラフィックのみを一時的に遅延させることで，MLS Commit メッセージを識別・破棄することなく鍵の非同期状態を誘発する．第二の制約に対して，DAVE の部分暗号化と H.265 の Filler Data を利用した ENV ポリグロットを構成することで，単一の認証済み暗号文から，標的と非標的にそれぞれ異なる攻撃者選択映像をエラーやノイズなく再生させる．これにより，先行研究で必要であった精密なネットワーク操作と非標的側の無音化をともに不要とし，鍵コミット性欠如を利用した攻撃を，実環境で現実的に成立する脅威へと発展させる．

- **TCP 遅延のみによる鍵非同期化の実現**：DAVE では，鍵更新を担う制御シグナリングとメディアパケットが，それぞれ TCP と UDP で転送される．この分離構造を利用し，標的宛てのインバウンド TCP のみを一時的に遅延させることで，TLS トラフィックの復号，MLS Commit メッセージの識別，およびパケットの選択的破棄を一切必要とせず，標的を旧鍵に留めながら他の受信者を新鍵へ移行させる．これにより，先行研究よりも大幅に限定されたネットワーク制御能力のみで，検知困難な攻撃時間窓を生成できることを示す．
- **受信者ごとに異なる映像を再生する曖昧な暗号文の構築**：意図しない鍵で復号した際に NAL ヘッダが破損すると，WebRTC の RTP デパケッタイザによってパケットが破棄されるという実システム固有の障壁を解決する．DAVE が一部の NAL ヘッダを暗号化せず AAD として保護する仕様と，H.265 の Filler Data がデコーダによって安全に無視される性質を組み合わせることで，WebRTC のパケット検証を通過する曖昧な暗号文を構成する．これにより，単一の暗号文から標的には映像 A を，他の受信者には映像 B をエラーやノイズなく同時に再生させる映像注入攻撃を実現する．
- **実機検証による実行可能性の実証**：Discord の実クライアントおよび WebRTC メディアパイプラインを用いた PoC 実装を構築し，評価を行う．実験では，切断や UI 警告を生じさせない 4.0 秒の攻撃時間窓を確認し，5 秒間の映像に対する攻撃用暗号文を 0.40 秒で生成できることを示す．
- **他システムへの汎用性の評価**：本攻撃の成立条件を 4 つのシステム設計要件に一般化し，Discord 以外のリアルタイム通話アプリケーションに対しても，同様の脅威が成立することを示す．これにより，本研究で明らかにした問題が Discord 固有ではなく，E2EE グループ通話システムに共通し得る構造的な脅威であることを示す．

1.2 責任ある開示

本研究で実証した攻撃および実証コードを，2026 年 5 月に責任ある開示のプロセスに基づき Discord 社へ報告した．本報告は Discord 社により公式に脆弱性として認定され，報奨金が付与された．また，本研究で提案した対策を含む修正案が現在検討されている．

2. 準備

2.1 DAVE プロトコルの概要

DAVE（Discord Audio/Video Encryption）プロトコルは，グループボイスチャンネルやグループ DM における多人数参加型のリアルタイム音声および映像通話において，エンドツーエンド暗号化（E2EE）を実現するために設計されたプロトコルである [1]．DAVE はメディア暗号化・復号とエポックベースの鍵管理を規定し，実装が *libdave* として公開されている [5].

DAVE におけるメディア転送は WebRTC 規格に準拠しており，暗号化された音声・映像ストリームは RTP（Real-time Transport Protocol）パケットとしてカプセル化され，WebRTC の Encoded Transform API を介してメディアパイプライン上で処理される [6].

© 2026 Information Processing Society of Japan

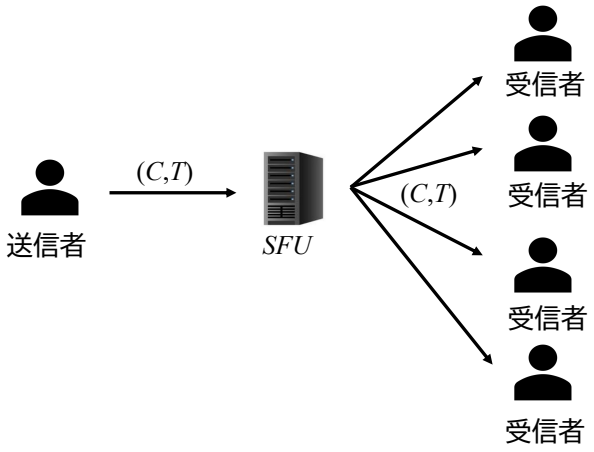


図 1 SFU によるブロードキャスト

メディアデータの暗号化には，認証付き暗号（AEAD）である AES-GCM が採用されている．AES-GCM の暗号化・認証処理は以下のように表される．

$$(C, T) = \text{AES-GCM}(K, N, A, P)$$

ここで，K は暗号鍵，N はノンス，A は追加認証データ（AAD），P は平文ペイロードである．出力として暗号文 C とタグ T の組を得る．DAVE では，各送信者が固有の送信鍵 K を用いてメディアストリームを暗号化する．この送信鍵 K は，グループ内で共有される MLS（Messaging Layer Security）[2] の状態から得られるベースシークレットを基に導出されるため，グループ内のすべての正当な参加者は同一のエポックにおいて送信者の鍵 K を同様に導出・共有し，復号に用いることができる．

2.2 DAVE における鍵管理と更新プロセス

2.2.1 鍵管理アーキテクチャと更新モデル

DAVE プロトコルが稼働する Discord の通話システムは，セッション管理を行う Voice Gateway とメディア転送を行う SFU（Selective Forwarding Unit）の二層構造によって構成されている [1].

1. Voice Gateway と SFU の役割分離

Voice Gateway は，MLS プロトコルに基づいた鍵交換メッセージの配送，メンバーの参加・退出に伴うセッション管理，およびユーザー認証を担う．これに対し，SFU は音声や映像などのメディアデータのルーティングのみを担当する．SFU は E2EE の復号鍵を保持しないため，コンテンツの内容を関知せず，暗号化されたパケットをグループ内の他の全受信者に対してブロードキャストする（図 1）.

2. 鍵更新と複数鍵の保持

グループ通話のメンバーシップ変更が発生すると，MLS のグループ状態は新しいエポックへと遷移し，新たなグループ鍵 K_{new} が生成される．しかし，リアルタイム通話環境は分散的かつ非同期に動作するため，全メンバーが同時に新しい鍵へと切り替えることはできない．ネットワーク遅延やメッセージの順序変更を許容し，通話の途切れを防ぐため，DAVE の復号クライアントは一時的に新旧複数の鍵を並行して保持する設計を採用している．具体的には，新しいエポックへの遷移後も，クライアントは最大 10 秒間，古いエポックの鍵 K_{old} を保持する．

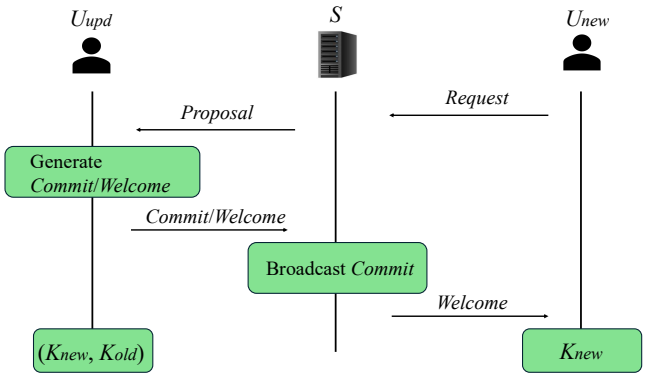


図 2 鍵更新の完了

3. 復号試行

DAVE のメディア暗号化レイヤーでは，AEAD の追加認証データ（AAD）に暗号化に用いた鍵を識別する KID（Key Identifier）が含まれない．そのため，受信側クライアントは受信した暗号パケットがどのエポックの鍵で暗号化されたかを事前に判定できない．受信側はまず最新の鍵 K_{new} で復号を試み，タグ検証に失敗した場合に古い鍵 K_{old} を用いて復号を試みる．

2.2.2 MLS に基づく鍵交換プロトコルの詳細

DAVE における鍵更新は，MLS プロトコル [2] に準拠して行われる．新規メンバー U_{new} が既存グループ（既存メンバー U_i，更新実行役 U_{upd}，サーバー S）に参加する際の鍵更新プロセスは，以下の通りである（図 2）.

(1) 提案と合意：U_{new} は鍵情報を含む参加リクエストを S へ送信し，S は提案（*Proposal*）を既存メンバーへ配送する．提案を受信した U_{upd} は，グループ状態を更新するための *Commit* メッセージおよび新メンバー用の *Welcome* メッセージを生成し，S へ送信する．

(2) 配信と状態更新：S は *Commit* を既存メンバー U_i へ，*Welcome* を U_{new} へそれぞれ配信する．これらを受信した各メンバーは，MLS の状態遷移を実行し，新たなエポックのメディア鍵 K_{new} を導出する．

(3) 複数鍵の保持：鍵導出時，クライアントは新鍵 K_{new} をアクティブな鍵リストに追加するとともに，古い鍵 K_{old} に最大 10 秒の有効期限を設定する．

2.3 鍵コミット性

鍵コミット性とは，暗号文 C と認証タグ T の組が，それを生成しうる暗号鍵 K に計算量的に一意に対応する性質を指す．Albertini ら [3] は，鍵コミット性に対する攻撃

© 2026 Information Processing Society of Japan

者のアドバンテージを以下のように定義している．

定義 1　鍵コミット性アドバンテージ [3]

攻撃者 $\mathcal{A}$ が行う q 回のクエリ Q_k $(k \leq q)$ において，以下の条件を満たす組 (Q_i, Q_j) が存在する確率．

$$K_i \neq K_j, \quad (N_i, C_i) = (N_j, C_j), \quad M_i \neq \perp, \quad M_j \neq \perp$$

ここで $\perp$ は復号エラーを表す．すなわち，異なる鍵 K_i, K_j を用いているにもかかわらず，同一のノンス N と暗号文 C が双方で受理（$M \neq \perp$）される場合，その方式は鍵コミット性を満たさない．

定義 2　曖昧な暗号文

異なる鍵 K_i, K_j $(K_i \neq K_j)$ において，同一のノンス N および追加認証データ A の下で，単一の暗号文 C が双方で正常に受理されるとき，この暗号文 C を曖昧な暗号文（Ambiguous Ciphertext [3]）と呼ぶ．

DAVE で採用されている AES-GCM は，認証タグ生成に用いる GHASH の代数的構造に起因して鍵コミット性を満たしておらず，攻撃者は方程式を解くことで曖昧な暗号文を効率的に生成できる [3]．

2.4　既存の音声注入攻撃とその課題

山下ら [4] の先行研究では，鍵コミット性欠如が実システム上で実際に危険となり得ることを示した．具体的には，MLS の Commit メッセージを選択的に遮断するネットワーク攻撃者と，音声コーデック Opus のデコード特性を組み合わせることで，特定の標的にのみ攻撃者が意図した音声を再生させ，他の参加者にはパケットロスによる無音として処理させる音声注入攻撃を提案した．

しかし，実環境で現実的な攻撃として成立させるには，以下の二つの技術的課題が残されていた．

1. 高度なネットワーク遮断の必要性

受信者間で鍵の非同期状態を強制するため，ネットワーク上で暗号化された通信の中から，特定の MLS Commit メッセージのみをパケットレベルで正確に識別し，標的への配送のみを破棄する必要があった．この高度な能力の要求が，攻撃の実行難易度を著しく高めていた．

2. 非標的側の出力制御の欠如

曖昧な暗号文から復号される非標的側の平文はランダムデータになるため，非標的受信者に対してはパケットを破棄させて無音にすることしかできなかった．すなわち，送信した単一の暗号文から，標的と他の受信者双方の画面やスピーカーに対し，それぞれ攻撃者が意図した異なるメディアを同時に再生させる攻撃は原理的に不可能であった．

3.　攻撃者モデルと安全性要件

3.1　攻撃者モデル

本稿で想定する攻撃者モデルを示すため，DAVE プロトコルにおいて規定されているエンティティの分類に基づき，攻撃主体の役割を以下のように定義する．

- A_{net}：標的ユーザー U_{tgt} の通信路上に位置する攻撃者であり，DAVE における *Outsiders* [1] に該当する．暗号化された TLS トラフィックを復号する能力や，改ざんする能力は持たず，標的宛ての TCP トラフィックを一時的に遅延させる能力に限定される．
- U_{mal}：グループ通話に参加する正当な権限を有し，*Participants* [1] に該当する．グループ内で更新された最新の鍵 K_{new} および過去のエポックにおける古い鍵 K_{old} を並行して保持する．また，AES-GCM の鍵コミット性欠如を利用して，双方の鍵で正常に検証が成功する曖昧な暗号文を生成・送信する．
- U_{trig}：グループ通話の入退室等により，エポック遷移（鍵更新）を自動的に誘発するユーザーであり，*Participants* [1] に該当する．

3.2　安全性要件

DAVE プロトコルでは，E2EE において保証されるべき主要な暗号学的要件として機密性（Confidentiality），完全性（Integrity），真正性（Authenticity）の 3 つを定義している [1]．これら 3 つの基本要件に加え，本稿では，E2EE を用いたグループ通信プロトコルが暗黙的かつ本質的に前提としている以下の信頼要件を定義する．

定義 3　参加者間の一貫性（Consistency）

セキュアなグループ通話において，同一の暗号文を受信した全参加者間で，復号・再生されるメディアの内容が同一となる性質．グループ鍵管理によりグループ状態が厳密に維持されている場合，受信者はこの性質を信頼できる．

4.　映像注入攻撃

本章では 2.4 節で示した先行研究における二つの技術的課題を解決し，実環境で成立する映像注入攻撃を提案し，その実証評価について述べる．

第一に，特定のメッセージを選択的に識別・遮断する必要があるというネットワーク上の課題に対し，一時的な TCP 遅延戦略を提案する．これにより，複雑なパケット解析や改ざんを伴わずに，より簡易かつ検知困難な方法で安全に鍵の非同期状態を誘発できる．第二に，非標的側の復号結果を制御できず無音化するしかないという障壁に対し，H.265 のコーデック仕様と DAVE の部分暗号化を利用し，単一の認証済み暗号文が復号鍵に応じて異なる有効な映像として解釈される曖昧な暗号文の構成手法を提案する．これにより，エラーやノイズを発生させずに，双方の受信者に異なる映像を同時にデコード・再生させる．

以下，これら二つのアプローチの具体的な設計と，実機検証の結果について述べる．

© 2026 Information Processing Society of Japan

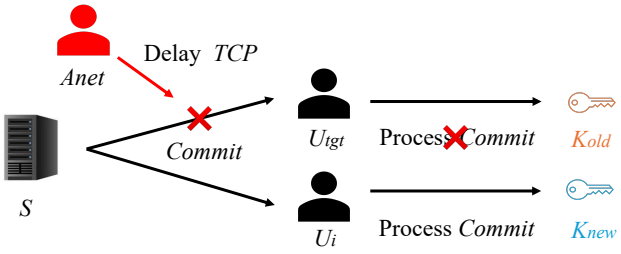


図 3 TCP 遅延による鍵非同期状態の誘発

4.1 一時的な TCP 遅延による鍵非同期化

4.1.1 制御トラフィックの遅延戦略

リアルタイムのグループ通話において受信者間に鍵の非同期状態を誘発するため，DAVE プロトコルにおける制御シグナリングとメディアデータのトランスポート層プロトコルの分離に着目する．DAVE では，参加者の状態制御や鍵更新を促すシグナリングメッセージは TCP を介して伝送される一方，暗号化されたメディアストリームは低遅延転送のために UDP を介して配信される．

ネットワーク上の攻撃者 A_{net} は，この分離構造を利用して，標的である U_{tgt} 宛てのインバウンド TCP フローを一時的に遅延させる一方で，UDP メディアパケットは遅延なく通過させる．これにより，他の受信者 U_i が最新鍵 K_{new} へと移行する間も，U_{tgt} 宛ての鍵更新メッセージが遅延されるため，U_{tgt} は旧鍵 K_{old} に留まる（図 3）．

4.1.2 遅延耐性の実機評価と攻撃時間窓

一時的な TCP 遅延操作が実際の E2EE グループ通話に与える影響を検証するため，複数の Discord クライアントを対象とした実機評価を行った．検証実験では，標的宛てのインバウンド TCP 通信に対してシミュレータによる意図的な遅延を注入し，鍵更新プロセスおよび通話セッション維持への影響を測定した．

実験の結果，遅延時間 X が 4.0 秒以下であれば，標的クライアントにおいて通話セッションの切断や接続品質に関する警告表示を一切伴わずに，新旧鍵の非同期状態を維持できることが明らかとなった．これにより，新旧双方の鍵で検証に成功する曖昧な暗号文 C^* を送信するための X 秒間の攻撃時間窓が生じる（図 4）．なお，実験環境，検証手順および詳細な測定結果は付録 A.1 に示す．

4.2 曖昧な暗号文の構築

4.2.1 H.265 デコーダのエラー耐性と単純な結合の限界

DAVE プロトコルにおいてサポートされる映像コーデックの一つである H.265 において，ビデオビットストリーム S は，3 バイトのスタートコード SC（0x000001）と NAL ユニットの繰り返しとして，以下のように定義される [7]．

$$S = SC \parallel NAL_1 \parallel SC \parallel NAL_2 \parallel \dots$$

実際の多くの H.265 デコーダは，ストリーム中に破損した

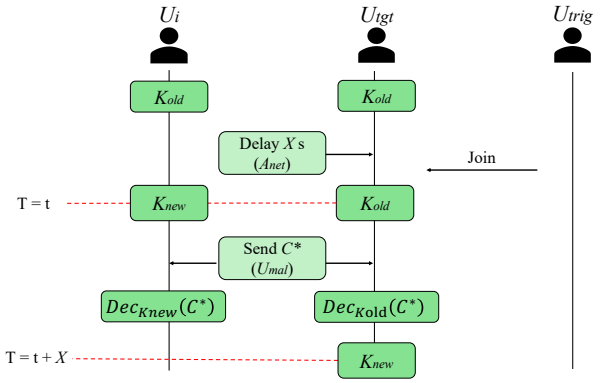


図 4 遅延による攻撃時間窓

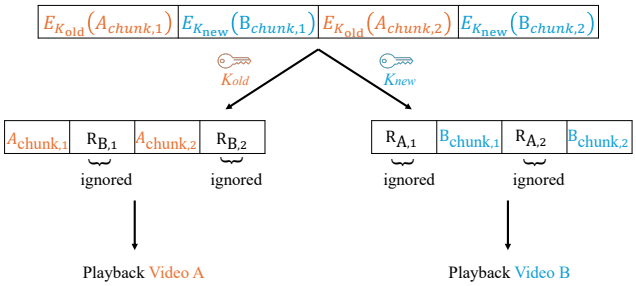


図 5 単純な結合による曖昧な暗号文の構成手法と再生結果

データ領域や不明なバイト列が存在する場合，それらを自動的にスキップし，次の有効な SC を検出するまでその領域を安全に無視するエラー耐性を備えている．

この性質を利用し，映像 A を旧鍵 K_{old} で，映像 B を新鍵 K_{new} でそれぞれ個別に暗号化した NAL ユニット（A_{chunk} および B_{chunk}）を交互に配置する，単純な結合データの構築手法が考えられる（図 5）．このデータを旧鍵 K_{old} で復号すると，映像 A の NAL ユニットは正常に復元されるが，映像 B の NAL ユニットはランダムなバイト列 R へと復号される．逆に，新鍵 K_{new} で復号した場合には映像 B のみが正常に復元され，映像 A がランダムなバイト列へと復号される．復号後に破損したデータ領域はデコーダのエラー耐性により無視されるため，ローカルなデコーダ上であればノイズを再生させることなく，鍵に応じた映像の非対称な再生が成立する．

しかし，この単純な結合構成を WebRTC を介したリアルタイム通信に適用する場合，重大な制約が生じる．WebRTC のメディアパイプラインにおいて，RTP デパケッタイザは，復号されたパケットをビデオデコーダに引き渡す前に，ペイロードの先頭に位置する 2 バイトの NAL ヘッダ領域を検証する [8]．上記の単純な結合では，意図しない鍵で復号された際にこのヘッダ領域までランダムなデータへと破損するため，RTP デパケッタイザによって不正なパケットと判定され，デコーダに到達する前にパケットが破棄される．したがってこの制約が，実際の通信環境における映像注入攻撃の障壁となる．

© 2026 Information Processing Society of Japan

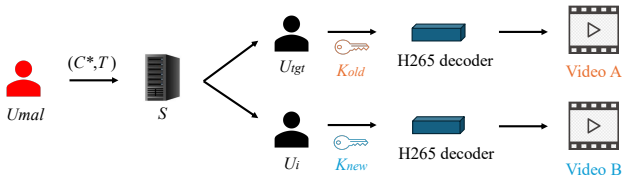


図 6　映像注入攻撃による非対称な再生結果の実現

4.2.2　DAVE の部分暗号化と H.265 の Filler Data を利用した ENV ポリグロットの構成

DAVE プロトコルのメディア暗号化における部分暗号化と，H.265 の Filler Data を組み合わせることで，前述した WebRTC のパケット破棄を回避する ENV ポリグロット構造を提案する．

DAVE では，リアルタイムなパケット処理のオーバーヘッド削減のため，Non-VCL NAL ユニット（`nal_unit_type` ≥ 32）の先頭 2 バイトの NAL ヘッダを暗号化せず追加認証データ（AAD）として保護する仕様を採用している [1], [5]．本手法では，この仕様と，ビットレート調整用のダミーデータをカプセル化するために定義されている Filler Data NAL ユニット（FD_NUT, `nal_unit_type` $= 38$ [7]）を利用する．暗号化されていない正当な FD の NAL ヘッダをパケットの先頭に配置し，これを後続データの外被（ENV：envelope）として用いる．その内部に鍵依存の衝突データおよび暗号化された実映像データを埋め込む．これにより，パケットは WebRTC の RTP デパケッタイザの検証を通過し，ビデオデコーダへと正常に配送される．

復号後のペイロードでは，使用する鍵に応じて一方の映像が正常に復元され，他方は破損して擬似ランダムなバイト列となる．この破損データは，Filler Data のボディ領域，およびデコーダのエラー耐性によりスキップされる領域に配置される．H.265 の仕様上，Filler Data のボディは映像出力に影響しないパディングデータであるため，デコーダは次の有効なスタートコードを検出するまで当該領域を安全に無視する [7]．この曖昧な暗号文を生成する具体的なアルゴリズムの詳細は，付録 A.2 に記述する．

我々は，実際の DAVE プロトコルに準拠した WebRTC メディアパイプライン上で PoC[*1] 実証実験を行い，この ENV ポリグロットを用いた映像注入の成立を実証した．図 6 に示すように，同一の暗号文ストリームを受信した際，暗号タグの検証はいずれの受信者でも成功し，旧鍵 K_{old} を保持する標的クライアントでは映像 A が，最新鍵 K_{new} を保持するクライアントでは映像 B が，ノイズや瞬断を伴うことなく正常かつ同時に再生される．

4.2.3　暗号文生成の計算コストと所要時間

提案する曖昧な暗号文の生成は，極めて高速かつ予測可能な時間で完了する．本手法の具体的な生成アルゴリズムの詳細は付録 A.2 に譲るが，計算コストの高い総当たり探索を一切必要とせず，アルゴリズム全体は入力データサイズ N に対して $\mathcal{O}(N)$ の線形時間で動作する．実機 PoC 評価では，二つの 5 秒間の映像ストリーム（合計 1.0MB）から ENV ポリグロット構造を持つ曖昧な暗号文を生成するのに要した時間はわずか 0.40 秒であった．

[*1] https://drive.google.com/file/d/1aWgiZs28OviHtMXTu5XPuHz0_R_8qUjx/view?usp=sharing

表 1　映像注入攻撃における時間設計

処理ステップ	実行主体	所要時間	参照元
入退室によるトリガー	U_{trig}	~ 0 秒	—
TCP 遅延の開始	A_{net}	~ 0 秒	—
新鍵エポック遷移の検知	U_{mal}	≤ 0.31 秒	付録 A.1.4
暗号文の生成（5 秒の映像）	U_{mal}	0.40 秒	4.2.3 節
SFU へのパケット転送・配信	U_{mal}	< 0.1 秒	[9]
合計所要時間	—	$\sim$ **0.81** 秒	—
利用可能な攻撃時間窓	—	**4.0** 秒	4.1.2 節
時間的猶予	—	**3.19** 秒	—

4.3　攻撃の評価

本節では，提案手法を組み合わせた攻撃全体の成立性，検知困難性，実環境への影響を評価する．

4.3.1　統合的な攻撃タイムラインの評価

本攻撃が実環境で成立するための時間的設計について評価する．各プロセスの所要時間と時間的猶予の測定結果を表 1 に示す．ここに示す数値は，実機ログ（付録 A.1.4）および 4.2.3 節の測定結果に基づいている．

表 1 の通り，新鍵エポックの遷移検知から暗号文生成，パケット配信までの所要時間は約 0.81 秒である．暗号文生成に要した 0.40 秒は 5 秒間の映像を仮定した値であり，注入映像が短ければさらに小さくなるため，0.81 秒は上界である．これは 4.1.2 節で示した切断や UI 警告を伴わない攻撃時間窓に対して十分に小さく，3.19 秒の猶予が残る．したがって一時的な処理遅延を考慮しても，本攻撃は高い確実性をもって実行可能である．

4.3.2　検知困難性の評価

提案手法の検知困難性を山下らの先行研究 [4] と比較して評価する．

第一に，ネットワーク層における課題を解決した．先行研究では，暗号化された制御トラフィックから特定の Commit メッセージを識別して能動的に遮断する必要があったが，提案手法は標的宛てのインバウンド TCP を一時的に遅延させるのみであり，パケットの識別を不要とし損失も伴わない．遅延解除後は標的が正規のエポックへ同期するため，鍵状態にも不整合が残らない（付録 A.1）．

第二に，暗号処理層では，曖昧な暗号文が新旧双方の鍵で認証タグ検証に成功するため，いずれの受信者でも復号失敗や認証エラーが生じず，ログに異常が残らない．

第三に，アプリケーション層に残っていた挙動の不整合を解消した．先行研究では非標的側が無音化するため不審

© 2026 Information Processing Society of Japan

に思われる余地があったが，ENV ポリグロット構造では，エラーやノイズを伴わずに異なる映像を同時に再生されるため，再生時点で受信者が不整合を知覚する契機がない．

以上より本手法は，ネットワーク層，暗号処理層，アプリケーション層のいずれにも検知の手がかりを残さない．

4.3.3 攻撃の現実性と影響

提案する映像注入攻撃が現実的な攻撃者モデルの下で成立し，深刻な影響をもたらし得ることを示す．

第一に，攻撃者モデルの現実性である．本攻撃で定義した各役割は，一人の攻撃者が複数アカウントとネットワーク制御権を持つことで統合できる．ネットワーク攻撃者 A_{net} に要求される操作は，暗号通信の改ざんや解析を伴わず，標的宛てのインバウンド TCP を一時的に遅延させるのみであり，Wi-Fi アクセスポイントやローカルルーターを制御できる攻撃者にとって容易である．

第二に，意思決定の欺瞞という影響である．単一の暗号文が双方の受信クライアントで正当なものとして受理されるため，参加者間で表示される映像が異なる状態を作り出せる．例えば企業の会議通話において，決定権を持つ標的には承諾を示すスライドを，他の参加者には却下を示すスライドを表示させ，当事者間の合意形成を破綻させることが可能となる．

すなわち本攻撃は，限定的なネットワーク制御能力のみで，同一の暗号文を受信した参加者間で再生されるメディアが一致しない状態を作り出す．これは 3.2 節の定義 3 で述べた参加者間の一貫性が DAVE において満たされていないことを意味する．MLS が保証するグループ状態の一貫性は，メディア暗号化層まで自動的には及ばない．

5. 他のアプリケーションへの応用

本稿で提案した映像注入攻撃は，特定の暗号プロトコル固有の実装バグではなく，E2EE リアルタイムグループ通話システム全般に共通しうる構造的な課題に根ざしている．本攻撃が一般に成立するための要件を，以下の 4 つの成立条件として整理する．

(1) **条件 1（鍵コミット性欠如）**：メディアデータの暗号化において，同一の暗号文が異なる鍵で正常に認証・復号され得る AEAD スキームを採用していること．

(2) **条件 2（非同期な鍵更新）**：グループ鍵管理において，ネットワーク遅延やメッセージの順序変更を許容するために，受信クライアントが一時的に新旧複数の鍵を並行して保持する設計であること．

(3) **条件 3（メディアフォーマットの冗長データ容認性）**：使用するメディアコーデックのデータ構造内に，破損データを埋め込める領域が存在し，デコーダがその領域を安全に無視・スキップできる特性を持つこと．

(4) **条件 4（認証付き鍵識別子の欠如）**：暗号化パケットに，使用する鍵を一意に特定する識別子（KID）が追加認証データ（AAD）等で暗号学的に紐付いておらず，受信側が複数鍵を試行する仕様であること．

表 2 E2EE アプリケーションの調査結果（条件 1–4）

アプリケーション	条件 1	条件 2	条件 3	条件 4
LINE	✓	∗	∗	×
Linphone	✓	✓	✓	✓
MS Teams	✓	–	–	–
Threema	✓	✓	✓	×
Wickr	✓	∗	∗	∗
Wire	✓	✓	✓	×
Zoom	✓	–	–	–
Element (Matrix)	×	∗	∗	∗
FaceTime	×	–	–	–
Facebook Messenger	×	–	–	–
Google Messages	×	–	–	–
iMessage (Apple)	×	–	–	–
KakaoTalk	×	–	–	–
Signal	×	∗	∗	∗
Viber	×	–	–	–
WhatsApp	×	–	–	–
Google Meet	–	–	–	–
Silent Phone	–	–	–	–
Skype	–	–	–	–
Telegram	–	–	–	–

✓：条件を満たす，×：条件を満たさない，–：公開情報が存在しない，∗：詳細な調査が必要．

これら 4 つの条件がすべて重なることで，実際のリアルタイム通信において参加者間の一貫性が侵害される．主要なリアルタイム E2EE 通話アプリケーションを対象に，これら 4 条件の適合性を調査・評価した（表 2）．

調査の結果，Linphone はこれら 4 つの条件をすべて同時に満たしていることが判明した．Linphone のグループ通話の E2EE は，鍵配布とメディア暗号化通信で構成される．鍵配布では通話の発信者が Signal プロトコルを通して参加者全員にメディアマスター鍵 ($mEkt$) を配信し，その後キーラップ関数 `AES-KW` と鍵 $mEkt$ を使用し，マスター暗号化鍵を参加者全員へ配布する．メディア暗号化通信ではまず，マスター暗号化鍵からメディアの種類 (映像，音声) ごとにセッション鍵を派生させる．派生したセッション鍵を用いて，メディアの暗号化を実施する．このときのメディア暗号化仕様は SRTP プロトコルに従う [10]．しかし，暗号パケットの受信側は KID の厳密な検証を行うことなく復号を実施する設計となっている．さらに，SRTP の KID 仕様である Master Key Identifier (MKI) は Linphone においては実装されていないため，条件 4 を満たしている．

なお, Linphone は RTP/SRTP の packetize-then-encrypt 順序に従うため，暗号化の粒度が RTP パケット単位となり，DAVE のようにフレーム全体を単一の鍵ストリームで覆うことができない．そのため意図しない鍵での復号で

© 2026 Information Processing Society of Japan

は NAL ヘッダを含むペイロード全体が擬似ランダム化され，デコーダに無視させる領域を平文として残せない．したがって，単一暗号文から同時に 2 つの異なる有効な映像を再生させる攻撃は困難である一方，K_{old} では正常に再生され K_{new} では無効なペイロードとして破棄される攻撃が成立する（付録 A.3.5）．

我々は，Linphone の実機上でグループ通話を確立し，一時的な鍵の非同期化を誘発させた上で，曖昧な暗号文を注入する PoC[*2]実証実験を行った．実験の結果，標的クライアントでのみ任意の映像を再生させ，他方のクライアントではアラートを発生させずにパケット破棄を誘発する，Discord と同種の攻撃が成立することを確認した．Linphone の鍵共有仕様，パケット構造，PoC 実験の詳細は付録 A.3 に記述する．

6. 修正案

本攻撃の根本原因は，複数の有効鍵を並行保持する仕様において，パケットが，どの鍵で暗号化されたかという文脈（鍵コンテキスト）と暗号学的に結びついておらず，かつ使用する AEAD が鍵コミット性を満たしていない点にある．この脆弱性への対処として，即時適用可能な短期的対策と，抜本的な長期的対策を提案する．

1. 短期的対策：追加認証データ（AAD）への鍵識別子の付加

メディアパケットの追加認証データ（AAD）に，使用する暗号鍵を一意に特定する鍵識別子（KID）を含めて暗号学的に紐付ける．受信クライアントは，受信パケットに対して無差別に復号試行を繰り返すのではなく，まず AAD 内の鍵識別子を検証し，意図された正しい鍵のみを選択して復号を実行する．これにより，意図しない鍵コンテキストにおいて同一の暗号文が正常に受理される経路を完全に遮断できる．この対策は Wire [11] などの他の主要な E2EE グループ通話システムでも既に採用されており，リアルタイム通信における帯域や計算のオーバーヘッドも無視できるほど小さい．

2. 長期的対策：鍵コミット性を有する AEAD への移行

暗号プリミティブのレイヤーにおいて，鍵コミット性を仕様レベルで数学的に保証する AEAD スキーム（CommitKey，DNDK [3], [12] など）へと移行する．ただし，リアルタイムのグループ通話環境における低遅延，高スループットといったパフォーマンス要件を満たす方式を選択する必要がある．

7. 結論

本稿では，鍵コミット性を欠く AEAD，非同期な鍵更新，コーデック仕様を組み合わせ，Discord の DAVE プロトコルにおいて参加者間の一貫性を破る映像注入攻撃を示した．TCP 遅延戦略と H.265 の Filler Data を用いた ENV ポリグロット構造により，単一の認証済み暗号文から標的と非標的にそれぞれ攻撃者が選択した異なる映像をノイズなく再生させた．

これは，精密なパケット遮断を要し非標的側を無音化するしかなかった先行研究の制約を取り除き，各層に痕跡を残さない現実的かつ検知困難な攻撃を初めて実現したものである．さらに，攻撃の成立条件を 4 つに一般化し，Linphone でも同様の侵害が再現可能であることも示した．

対策としては，短期的には AAD への鍵識別子の付加，長期的には鍵コミット性を有する AEAD への移行が有効である．本研究の成果は，E2EE の安全性が暗号プリミティブ単体ではなく，トランスポートやコーデックを含むシステム全体の設計の一貫性に依存することを示している．

謝辞

本研究は，JST CRONOS JPMJCS25N1，JSPS 科研費 JP24H00696 および JST ACT-X JPMJAX25M8 の支援を受けたものである．

[*2] https://drive.google.com/file/d/1ZL_NudFGaM_dob49ewrpLVTu4vwauK4r/view?usp=sharing

参考文献

[1] Discord Inc.: Discord DAVE Protocol Whitepaper, https://daveprotocol.com/ (2024).
[2] Barnes, R., Beurdouche, B., Robert, R., Millican, J., Omara, E. and Cohn-Gordon, K.: The Messaging Layer Security (MLS) Protocol, RFC 9420 (2023).
[3] Albertini, A., Duong, T., Gueron, S., Kölbl, S., Luykx, A. and Schmieg, S.: How to Abuse and Fix Authenticated Encryption Without Key Commitment, *USENIX Security* (2022).
[4] 山下 勇斗，田中 篤，木村 隼人，五十部 孝典: 鍵コミット性欠如を利用した Discord のエンドツーエンド暗号化に対する不正音声再生攻撃，暗号と情報セキュリティシンポジウム (SCIS2026).
[5] Discord Inc.: libdave, https://github.com/discord/libdave (2026).
[6] W3C: WebRTC Encoded Transform (2024).
[7] ITU-T: High efficiency video coding (2019).
[8] Wang, Y.-K., Sanchez, Y., Schierl, T., Wenger, S. and Hannuksela, M. M.: RTP Payload Format for High Efficiency Video Coding (HEVC), RFC 7798 (2016).
[9] André, E., Le Breton, N., Lemesle, A., Roux, L. and Gouaillard, A.: Comparative Study of WebRTC Open Source SFUs for Video Conferencing, *IPTComm*, pp. 1–8 (2018).
[10] Belledonne Communications SARL: Secure Communications using Linphone & Flexisip, Technical report (2025).
[11] Wire GmbH: Wire Security Whitepaper (2025).
[12] Gueron, S. and Ristenpart, T.: DNDK: Combining Nonce and Key Derivation for Fast and Scalable AEAD, Cryptology ePrint Archive, Paper 2025/785 (2025).
[13] jagt: clumsy, https://jagt.github.io/clumsy/ (2023).
[14] Google: Chrome DevTools, https://developer.chrome.com/docs/devtools/.

© 2026 Information Processing Society of Japan

研究倫理

本研究は，責任ある開示および協調的な脆弱性開示の原則に従って実施した．実験の実施にあたっては実ユーザーの安全とプライバシーを優先し，対象サービスの本番環境や実ユーザーに対する攻撃は一切行っていない．すべての実験は我々自身のアカウントと端末のみを用いたローカルな閉環境で実施しており，他のユーザーのデータへのアクセス，サービス基盤の妨害はいずれも行なっていない．また分析は公開情報，ホワイトペーパー，およびオープンソースコードのみに基づき，非公開のサーバ側実装に対するリバースエンジニアリングは試みていない．

以下に，本稿で示した映像注入攻撃について，各サービスへの開示の経緯を示す．

Discord

2026 年 5 月に DAVE プロトコルに対する映像注入攻撃を報告し，同月に Discord から受領および速やかに確認するとの連絡を得た．2026 年 6 月，同社は本件を脆弱性として認定し，我々が提案した修正案を検討する旨および報奨金を付与する旨を連絡した．2026 年 7 月に報奨金を受領した．執筆時点においても，リスク評価と緩和策に関する議論を継続している．

Linphone

2026 年 5 月に E2EE グループ通話機能に対する映像注入攻撃を報告した．その後，開発チームより，報告内容を管理システムに登録し暗号技術の専門家が精査する予定である旨の返信を得た．執筆時点では，この解析結果に対する回答を待っている段階である．

付　　録

A.1 鍵非同期化実験および遷移時間測定の手順と結果

本付録では 4.1.2 節で示した一時的な TCP 遅延操作が実際の Discord クライアントに与える影響を検証するための実験環境と再現手順，およびその挙動の分類について述べる．さらに 4.3.1 節で示した攻撃タイムラインの根拠となる，鍵更新に伴う処理時間の測定実験の手順と結果を示す．

A.1.1 実験環境および機材要件

グループ通話環境を再現するため，役割に応じた 3 つの Discord アカウントと個別の端末を用意した．

- **標的クライアント（U_{tgt}）**：Windows 端末．インバウンド TCP トラフィックのみを遅延させるため，ネットワークシミュレータ clumsy（v0.3） [13] を動作させる．Chrome ブラウザ版（DevTools [14] による内部 MLS 状態のログ監視のため推奨）および公式デスクトップアプリケーション版の双方を使用する．
- **制御・監視メンバー（U_i）**：任意の OS 端末．Chrome ブラウザ版で常駐し，標的へ向けたビデオストリームを送信するカメラソースの役割を果たす．
- **トリガーメンバー（U_{trig}）**：任意の OS 端末．通話セッションへの入退室を繰り返すことで，DAVE サーバー側に動的な MLS の Commit メッセージの配信を誘発させる．

A.1.2 ネットワーク遅延シミュレーション手順

状態制御メッセージの干渉を排除するため，マイクやカメラの ON/OFF 状態を事前に固定した上で，以下の手順で実験を行う．

(1) **初期化と安定化**：U_{tgt} と U_i の間でビデオ通話を確立し，双方のマイクをミュート，監視メンバー U_i のカメラのみを ON にして，ビデオストリームが安定して流れている状態にする．

(2) **遅延の注入**：標的 U_{tgt} 上で clumsy を起動し，以下のパラメータを設定して遅延注入を開始する．

- **Filter**：`tcp and inbound`（UDP メディアには干渉せず，鍵更新処理を行う TCP トラフィックのみを遅延させる）
- **Function**：`Lag`（遅延時間 X 秒．主要な測定値として 4.0 秒，5.0 秒，10.0 秒などを検証）

(3) **鍵更新の誘発**：遅延が有効な状態で，トリガーメンバー U_{trig} を通話セッションに参加または退出させ，鍵更新用の Commit メッセージを発生させる．

(4) **観察**：遅延解除後にクライアントが切断されることなく自動復旧するか，および UI 上に警告等が表示されるかを観察・記録する．

A.1.3 実験結果と挙動の分類

遅延時間 X の違いに応じて，標的クライアントは以下の 3 つの挙動に分類されることが確認された．

- **$X \leq 4$ s の場合**：遅延中，標的のクライアントは新しい鍵を確定するメッセージを受け取れないため，古い鍵を使用する．他のメンバーは新しい鍵に切り替えるため，標的の画面では相手の映像が一時的に停止する．しかし，遅延が解除されると，警告や切断の表示を出すことなく自動的に正常な通話に復帰する（`TCP_Delay_4s_Recovery.mp4`[*3]）．
- **5 s $\leq X \leq$ 10 s の場合**：ブラウザ版のクライアントでは，遅延が 5.0 秒を超えるとタイムアウトと判定され，通話が一度切断されて再接続の画面が表示される（`TCP_Delay_5s_Disconnect.mp4`[*4]）．一方で，デスク

[*3] https://drive.google.com/file/d/1pqPvyjIEn3XOIfOKexRYtEcMKKkB2m9h/view?usp=sharing

[*4] https://drive.google.com/file/d/18fvY9pO_V6oTPT4190126zIbuUuWkki8/view?usp=sharing

© 2026 Information Processing Society of Japan

トップアプリ版では，この長さの遅延でも切断されずに自動で復旧するという違いが見られた．

- $X \geq 10$ **s の場合**：Discord の仕様により，他のメンバーが持つ古い鍵の保存期限である 10.0 秒を超えるため，標的が送る映像データを他のメンバーが受け取れなくなる不整合が残る．

通信の種類を制限せずすべてのデータを 4.0 秒間遅延させた対照実験では，映像データそのものが遅れるため，画面に品質低下を示す警告表示が直ちに出現し，ユーザーに察知される（`TCP_UDP_Delay_Warning.mp4`[*5]）．この対照実験から，鍵更新を担う TCP のみを遅延させる手法が検知困難性の維持に有効であることが確認できる．

A.1.4　鍵の更新処理にかかる時間の測定実験

メンバーの入退室を 39 回繰り返して意図的に鍵の更新を発生させ，システムが新しい鍵へ切り替えるのに要する時間を Chrome のコンソールログから測定する実験を行った[*6]．測定された各処理の所要時間は以下の通りである．

- **更新の提案メッセージを受け取ってから新しい鍵が有効になるまでの所要時間**：最小 154 ms，平均 220 ms，最大 307 ms
- **更新の確定メッセージを受け取ってから新しい鍵が有効になるまでの所要時間**：最小 52 ms，平均 98 ms，最大 197 ms

この実験結果から，鍵の切り替えにかかる時間は最大でも 0.31 秒で完了することが確認された．

A.2　曖昧な暗号文生成アルゴリズムの詳細

4.2.2 節で述べた ENV ポリグロット構成による曖昧な暗号文の具体的な生成手順を，アルゴリズム 1 に示す．本節では，各ステップが担う役割と設計上の意図を述べる．

ステップ 1–2：ENV の宣言と平文の整列

まず，H.265 の Filler Data NAL ユニットのスタートコードと NAL ヘッダ（`0x000001` ∥ `0x4C01`，`nal_unit_type` = 38）を非暗号化領域として先頭に配置し，これをそのまま AAD として宣言する．これにより，RTP デパケッタイザが検証する先頭 2 バイトの NAL ヘッダは，いずれの鍵で復号した場合にも常に正当な値として保たれる．続いて，映像 A のチャンク A と映像 B のチャンク B を連結し，AES-GCM のブロック長に合わせて 16 バイト境界までゼロパディングした P_{base} を平文とする．ブロック境界への整列は，後段の GHASH 衝突をブロック単位で解くために必要である．

ステップ 3：クロス鍵ストリーム配置とスタートコード回避

暗号文本体は，領域ごとに異なる鍵の鍵ストリームを用いる交差配置によって構成する．すなわち，映像 A に対応する領域は旧鍵 K_{old} から導出した鍵ストリームで，映像 B に対応する領域は新鍵 K_{new} から導出した鍵ストリームでそれぞれマスクする．この結果，同一の暗号文 C^* は K_{old} で復号すると映像 A を，K_{new} で復号すると映像 B を復元し，各鍵において意図しない側の領域は擬似ランダムなバイト列となる．これらの破損領域は Filler Data のボディ領域内，またはデコーダのエラー耐性によってスキップされる領域に位置するため，デコーダは次の有効なスタートコードを検出するまで当該領域を安全に読み飛ばす．

さらに，末尾の 16 バイトをダミーブロックとして予約し，GHASH 衝突を解いて得られる調整ブロックで置換する．GHASH は鍵ごとに定まるハッシュ副鍵上の線形写像であるため，1 ブロック分の自由度を用いることで，新旧双方の鍵において認証タグが一致する調整ブロックを解くことができる [3]．

ただし，交差配置によって生じる擬似ランダム領域および調整ブロックには，スタートコード `0x000001` が偶発的に出現し得る．デコーダがこれを誤った NAL ユニット境界として認識すると，読み飛ばすべき破損領域が映像データとして解釈されてしまう．そこでノンスを *attempt* だけ変化させながら再構成を試行し，最終的な暗号文 C^*，および K_{old}・K_{new} それぞれで復号した際の破損領域のいずれにもスタートコードが含まれないことを確認するループを設ける．長さ N バイトの擬似ランダム領域にスタートコードが出現する確率は高々 $N \cdot 2^{-24}$ であり，本評価の入力サイズでは十分小さい．したがってこの試行は通常数回で収束する．

ステップ 4–5：認証タグの確定とパケットの組み立て

スタートコードを含まない C^* が得られた後，$J_0 = IV \parallel$ `0x00000001` を用いて GCM の認証タグを計算する．ステップ 3 で調整ブロックを解いているため，このタグは新旧双方の鍵で検証に成功する．DAVE の仕様に合わせてタグを先頭 8 バイトに切り詰めるが，本攻撃は 16 バイトの標準的なタグ長でも同様に成立する．最後に DAVE のフッタを付与し，$ENV \parallel C^* \parallel Footer$ をパケットとして送出する．

本アルゴリズムの主要な処理は GHASH 衝突の計算とスタートコード回避ループであり，いずれも入力サイズ N に対して線形時間 $O(N)$ で動作する．

A.3　Linphone グループ通話 E2EE の仕様と攻撃の詳細

5 章で述べた Linphone への攻撃について，グループ通話 E2EE の鍵共有仕様，内部暗号化のパケット構造，および実機 PoC 実験の技術的詳細を示す．Linphone のグループ通話 E2EE は公式に文書化されていないため，本節の記

[*5] `https://drive.google.com/file/d/1uwc-k1ouOyrpd9c3MwOqgw-ATE9r_RPR/view?usp=sharing`
[*6] `https://drive.google.com/drive/folders/1BoHBdGnvlKVY35VwaP1vCVXr-E3hv0fv?usp=drive_link`

© 2026 Information Processing Society of Japan

アルゴリズム 1 ENV ポリグロット構成を利用した曖昧な暗号文の生成

入力： 映像 A のチャンク A, 映像 B のチャンク B, 鍵 K_{old}, K_{new}, 基底ノンス IV_{base}，フレーム ID id
出力： 攻撃用映像パケット $ENV \parallel C^* \parallel Footer$

ステップ 1：ENV と AAD の構築
$ENV \leftarrow$ `0x000001` $\parallel$ `0x4C01`
$AAD \leftarrow ENV$
ステップ 2：平文の整列
$P_{base} \leftarrow \mathrm{pad16}(A \parallel B)$
ステップ 3：スタートコード回避ループ
for $attempt = 0$ **to** $maxRetry - 1$ **do**
  $IV \leftarrow (IV_{base} \oplus id) + attempt$
  $KS_{old} \leftarrow$ AES-CTR-KS$(K_{old}, IV, |P_{base}| + 16)$
  $KS_{new} \leftarrow$ AES-CTR-KS$(K_{new}, IV, |P_{base}| + 16)$
  $C_{tmp} \leftarrow (A \oplus KS_{old}[0{:}|A|])$
    $\parallel (B \oplus KS_{new}[|A|{:}|A|{+}|B|])$
    $\parallel KS_{new}[|A|{+}|B|{:}|P_{base}|] \parallel 0^{16}$
  $j \leftarrow$ index of the trailing 16-byte block of C_{tmp}
  $C^*[j] \leftarrow$ SolveGHASH$(C_{tmp}, AAD, K_{old}, K_{new})$
  $C^* \leftarrow C_{tmp}$ with block j replaced by $C^*[j]$
  $N_{old} \leftarrow C^* \oplus KS_{old}$
  $N_{new} \leftarrow C^* \oplus KS_{new}$
  if `0x000001` appears in none of C^*, $N_{old}[|A|{:}]$, $N_{new}[0{:}|A|]$, $N_{new}[|P_{base}|{:}]$ **then**
    break
  end if
end for
ステップ 4：認証タグの確定
$J_0 \leftarrow IV \parallel$ `0x00000001`
$T \leftarrow \mathrm{GHASH}(C^*, AAD, K_{old}) \oplus \mathrm{AES}(K_{old}, J_0)$
$T_{trunc} \leftarrow T[0{:}8]$
ステップ 5：パケットの組み立て
$Footer \leftarrow$ Build-DAVE-Footer$(T_{trunc}, IV, \dots)$
return $ENV \parallel C^* \parallel Footer$

述は公開ソースコードおよび技術資料 [10] から再構成したものである．

A.3.1 グループ通話 E2EE の全体構成

Linphone のメッセージングおよび 1 対 1 通話は，LIME ライブラリによる Signal プロトコルと，SAS 検証を伴う ZRTP という標準的な E2EE 機構を用いる．一方でグループ通話は独自設計であり，参加者に配布した鍵素材を用いて SRTP による内部暗号化（inner-encryption）でメディアを保護する．我々はこれを，(1) 鍵配布鍵 $mEkt$ の共有プロトコル，(2) 内部暗号化用マスター鍵の配布プロトコル，(3) 内部暗号化プロトコルの 3 つの構成プロトコルとして整理した（図 A·1）．

なお，1 対 1 通話の暗号化チャネル（outer-encryption）は，既定設定では E2EE が保証されず，またグループ通話に対して SAS 認証が提供されないため，メディアを中継するサーバに対する秘匿性・完全性を与えるものとは見なせない．したがって本稿では outer-encryption を考慮の対象外とする．

A.3.2 鍵配布鍵 $mEkt$ の共有と内部マスター鍵の配布

まず，鍵素材を配布するための鍵 $mEkt$ を共有する．$SSPI$ はサーバが生成する $mEkt$ の鍵識別子，$CSPI$ は代表参加者が生成し後に内部暗号化のマスターソルトとして用いられる乱数である．鍵 MK の導出と $mEkt$ の AES-GCM 暗号化は代表参加者の端末上で行われる．

$$SSPI \xleftarrow{\$} \{0,1\}^{16}, \tag{A.1}$$

$$CSPI \xleftarrow{\$} \{0,1\}^{128}, \tag{A.2}$$

$$mEkt \xleftarrow{\$} \{0,1\}^{256}, \tag{A.3}$$

$$MK \leftarrow KDF(KeyMaterial_{DR}), \tag{A.4}$$

$$MK_{key}, MK_{iv} \leftarrow MK[0...32], MK[32...48], \tag{A.5}$$

$$(C, T) \leftarrow \mathsf{AES\text{-}GCM.Enc}(MK_{key}, MK_{iv}, P = mEkt, AAD = SSPI \parallel CSPI \parallel DRHeader), \tag{A.6}$$

$$DRmsg \leftarrow DRHeader \parallel C \parallel T \tag{A.7}$$

次に，送信者は内部暗号化用のマスター鍵 $inner_master_key_s$ を乱数から生成し，$mEkt$ を鍵として AES-KW で暗号化する．Linphone の実装では，RTP セッション識別子 $SSRC$ およびロールオーバカウンタ ROC もマスター鍵と併せて暗号化され，鍵世代を表す $epoch$（初期値 0，更新ごとに 1 加算）などの平文メタデータとともに EKT_{Tag} として各参加者に配布される．

$$SSRC \xleftarrow{\$} \{0,1\}^{32}, \quad ROC \leftarrow \{0\}^{32}, \tag{A.8}$$

$$inner_master_key_s \xleftarrow{\$} \{0,1\}^{256}, \tag{A.9}$$

$$Plain \leftarrow Len \parallel inner_master_key_s \parallel SSRC \parallel ROC, \tag{A.10}$$

$$EKTCipher \leftarrow \mathsf{AES\text{-}KW.Enc}(K = mEkt, P = Plain), \tag{A.11}$$

$$EKT_{Tag} \leftarrow EKTCipher \parallel SSPI \parallel epoch \parallel len \parallel type \tag{A.12}$$

A.3.3 内部暗号化のパケット構造

内部暗号化には SRTP が用いられ, 既定の暗号スイートは AES-256-GCM である．実装は `libsrtp` に基づく．SEQ は 2 バイトの RTP シーケンス番号であり，$SRTP\text{-}KDF$ は SRTP 仕様で定義される AES カウンタモードによる鍵導出関数である．

$$salt_{master} \leftarrow CSPI[0...12], \tag{A.13}$$

$$ikm \leftarrow inner_master_key_s(32) \parallel salt_{master}(12), \tag{A.14}$$

$$K_{enc} \leftarrow \mathsf{SRTP\text{-}KDF}(M = ikm, label = \texttt{0x00}, L = 32), \tag{A.15}$$

© 2026 Information Processing Society of Japan

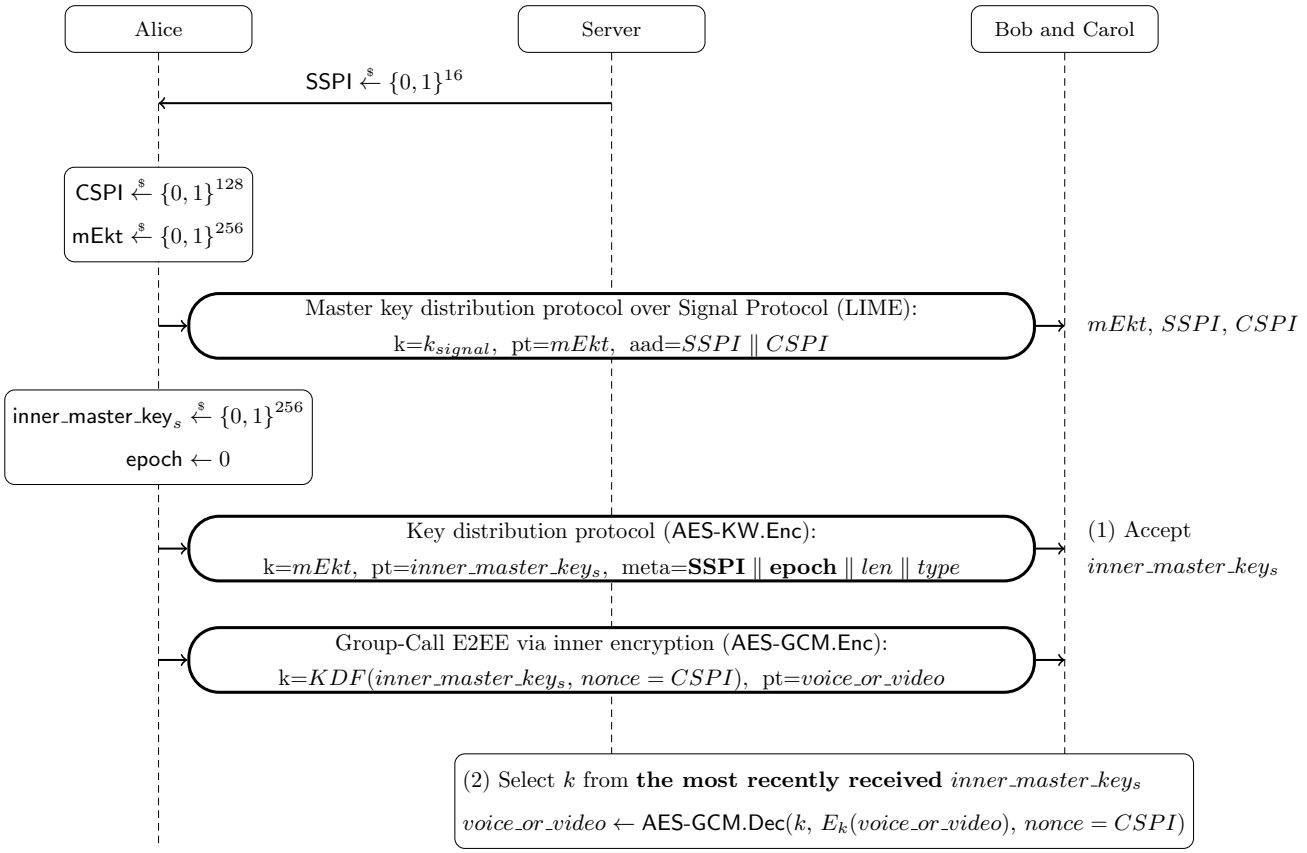


図 A·1 Linphone グループ通話 E2EE プロトコル

$$S_{sess} \leftarrow \mathsf{SRTP\text{-}KDF}(M = ikm, label = \mathtt{0x02}, L = 12), \tag{A.16}$$

$$in_{nonce} \leftarrow \mathtt{0x0000}(2) \parallel SSRC(4) \parallel ROC(4) \parallel SEQ(2), \tag{A.17}$$

$$nonce \leftarrow in_{nonce} \oplus S_{sess}, \tag{A.18}$$

$$(C, T) \leftarrow \mathsf{AES\text{-}GCM.Enc}(k = K_{enc}, n = nonce, P = data_{voip}, aad = Header_{RTP}), \tag{A.19}$$

$$packet \leftarrow C \parallel T \parallel Header_{RTP} \tag{A.20}$$

重要な点は，AAD が RTP ヘッダのみであり，どの鍵で暗号化されたかを示す情報が一切含まれないことである．SRTP にはこの検証のための Master Key Identifier（MKI）が任意機構として用意されているが，Linphone はこれを使用しない．受信側は送信者ごとに最後に受信したマスター鍵をそのまま用いて復号を行い，パケットが実際にどの鍵で暗号化されたかを検証しない．これが 5 章の条件 4（認証付き鍵識別子の欠如）に該当する．

A.3.4 鍵の非同期状態の誘発

DAVE の MLS エポック遷移と異なり，Linphone では参加者が自身の送信メディア鍵を任意に更新して鍵素材を再配布できる．攻撃者 U_{mal} はこれを鍵更新のトリガーとして利用する．U_{mal} が送信鍵を K_{old} から K_{new} へ更新した後，A_{net} は標的 U_{tgt} 宛ての鍵更新トラフィックのみを遅延させ，他の参加者へは遅延なく到達させる．これにより DAVE の場合（4.1 節）と同様に U_{tgt} は K_{old} に留まり，他の参加者は K_{new} へ移行するため，U_{mal} は受信者ごとに異なる鍵で受理される単一の暗号文 C^* を送信できる状態を得る．

A.3.5 packetize-then-encrypt 構造に起因する制約

DAVE との本質的な差異は暗号化の粒度である．DAVE はパケット化前に暗号化するため，単一の鍵ストリームがフレーム全体を覆い，これが H.265 の ENV ポリグロット構成（4.2.2 節）が利用するフレーム単位の自由度を与える．一方 Linphone は RTP/SRTP の packetize-then-encrypt 順序に従い，各 RTP ペイロードをパケットごとのノンスで独立に暗号化する．したがって暗号化の単位は RTP パケットであり，フレームが複数パケットに分割される場合にはフレームより細かい粒度となる．

この結果，意図しない鍵での復号では NAL ヘッダを含むペイロード全体が擬似ランダム化されるため，K_{new} の下で第二の有効な映像を成立させることはできない．すなわち Linphone に対する攻撃は，DAVE のような 2 つの有効な映像を同時に再生させる構成ではなく，K_{old} では正常に再生され K_{new} では不正なペイロードとして破棄される構成となる．

パケットごとに鍵ストリームと認証タグが 1 組ずつ存在

© 2026 Information Processing Society of Japan

表 A·1 パケット化方式ごとの攻撃成立性と計算コスト

パケット化方式	調整ブロック /NAL	タグ数 /フレーム	攻撃
Single NAL Unit	1	≥ 1	✓
Aggregation Packet	≤ 1	1	✓
Fragmentation Unit	> 1	> 1	×

いずれの列も値が大きいほど攻撃は困難となる．NAL あたりの調整ブロックが 1 を超えると配置が破綻し攻撃は成立しない．フレームあたりのタグ数が 1 を超えると暗号文生成コストが増大する．

するため，衝突計算に用いる擬似ランダムな 16 バイトの調整ブロックもパケットごとに 1 つ固定される．この調整ブロックは，K_{old} の下でデコーダが無視する領域に配置されなければならない．Single NAL パケットおよび集約パケットでは NAL あたり高々 1 つのタグしか存在しないため，調整ブロックを Filler Data 領域に収めることが容易である．一方，1 つの NAL を複数パケットに分割する Fragmentation Unit では，再構成後の NAL 内に断片ごとの調整ブロックが現れ，そのうち無視可能な末尾に収まるのは 1 つのみであるため，残りがデコーダの解析対象バイトに落ち，K_{old} 側の映像まで破損させる（表 A·1）．ただし攻撃者は送信者自身であるため，NAL を MTU 内に収めるか集約パケットを用いることでこの事態を回避できる．

A.3.6 PoC 実験と結果

我々は，E2EE を有効にした Linphone の実機グループ通話環境において，本節で述べた攻撃の統合的な PoC 実証実験を行い，攻撃が成立することを確認した．PoC[*2] は，E2EE が有効なグループ通話を確立し，A.3.4 節の手順で受信者依存の鍵状態を誘発した上で，稼働中のメディアストリームに曖昧な SRTP パケットを注入するものである．

実験の結果，標的の受信者は K_{old} で復号して攻撃者の意図した映像を再生する一方，他の受信者は同一の認証済みパケットを K_{new} で復号するが，復号可能な映像を得られないことを確認した．これにより，認証を伴わないパケット注入と比較して，実用上の検知困難性において優位となる．Linphone では暗号検証の失敗は UI の接続品質インジケータに反映されるが，認証成功後の映像デコード失敗は反映されない．そのため非標的の受信者には，認証失敗や受信品質の著しい劣化を示す明示的な警告を伴わず，短時間の映像停止やフレーム静止のみが観測される．

© 2026 Information Processing Society of Japan